

Proyecto de programación en ensamblador

Estructura de Computadores

- El proyecto consiste en la programación, en ensamblador del Motorola 88110, de un conjunto de rutinas que realicen la *compresión y descompresión de un texto almacenado en memoria*.
- El texto será una cadena de caracteres *ascii* almacenados en posiciones consecutivas de la memoria y que finaliza con el carácter *nul*. Cada carácter *ascii* es un *byte que se interpreta sin signo*, teniendo por lo tanto un valor comprendido entre **0** y **255**. El valor **0** corresponde al terminador, *nul*, y se representa mediante la secuencia `\0`
- El proceso de compresión se basa en la búsqueda de subcadenas de caracteres repetidas, de las que se mantiene la primera ocurrencia y se sustituyen las demás por referencias a la primera.

Enunciado curso 2025/2026

- **Coincidente con el enunciado de 2024/25.**

Al tratarse del mismo enunciado del curso pasado, aquellos alumnos que tengan que repetir o corregir el proyecto que desarrollaron entonces, podrán utilizar los programas ya realizados anteriormente por ellos mismos (manteniendo el mismo grupo).

- **Implementación y pruebas.**

Todas las rutinas deben ser probadas exhaustivamente para determinar que su funcionamiento es correcto y acorde con su especificación. Para superar el proyecto se requiere superar cierto número de pruebas de algunas subrutinas. En algunos casos, esas pruebas podrán verse modificadas a lo largo del curso.

- **Normas de entrega y evaluación.**

Es importante observar con atención el apartado del enunciado que describe las normas de entrega y evaluación. En particular, la evaluación de la parte de pruebas, la especificación del contenido que debe incluir la memoria del proyecto y las fechas de entrega de los *hitos evaluables*.

Enunciado curso 2025/2026

- **Formación de grupos.**

Los alumnos que ya hubieran formado parte de un grupo durante convocatorias anteriores y hubieran realizado al menos una entrega, solo podrán establecer grupo con el mismo compañero de dicha convocatoria o, alternativamente, registrarse y realizar el proyecto de forma individual.

- **Comprobación de casos de copia.**

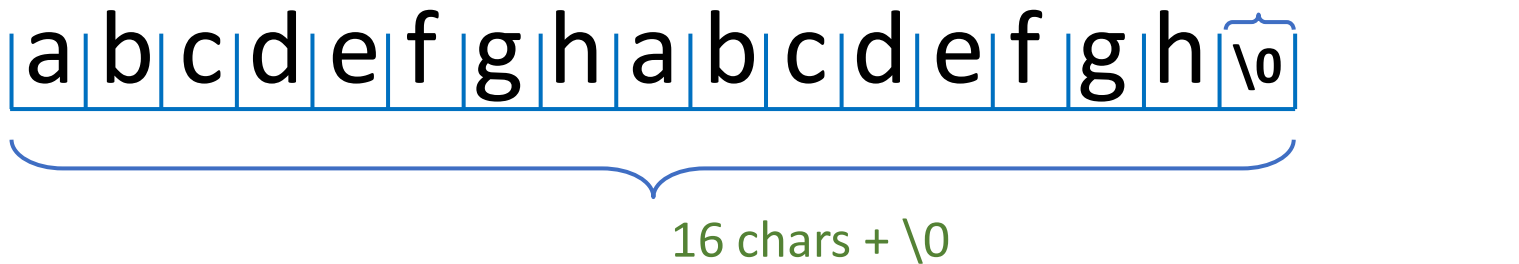
Se realizará una revisión minuciosa de los proyectos realizados en el curso actual y se verificarán sus posibles coincidencias con los otros proyectos realizados en este mismo curso o en los anteriores, de modo que pueda descartarse o localizar posibles casos de copia. Desafortunadamente, se siguen produciendo y detectando muy frecuentemente.

En relación con este aspecto se requiere que cada alumno presentado al proyecto incluya en la memoria una declaración de conocimiento de las normas de copia tal como la que se propone en el apartado correspondiente del enunciado.

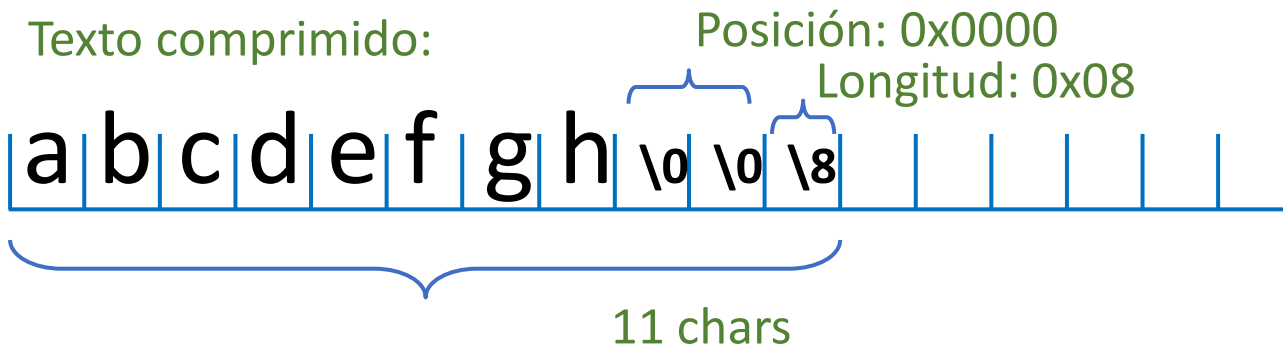
Compresión de texto

Ejemplo simplificado de compresión de un texto

Texto original:

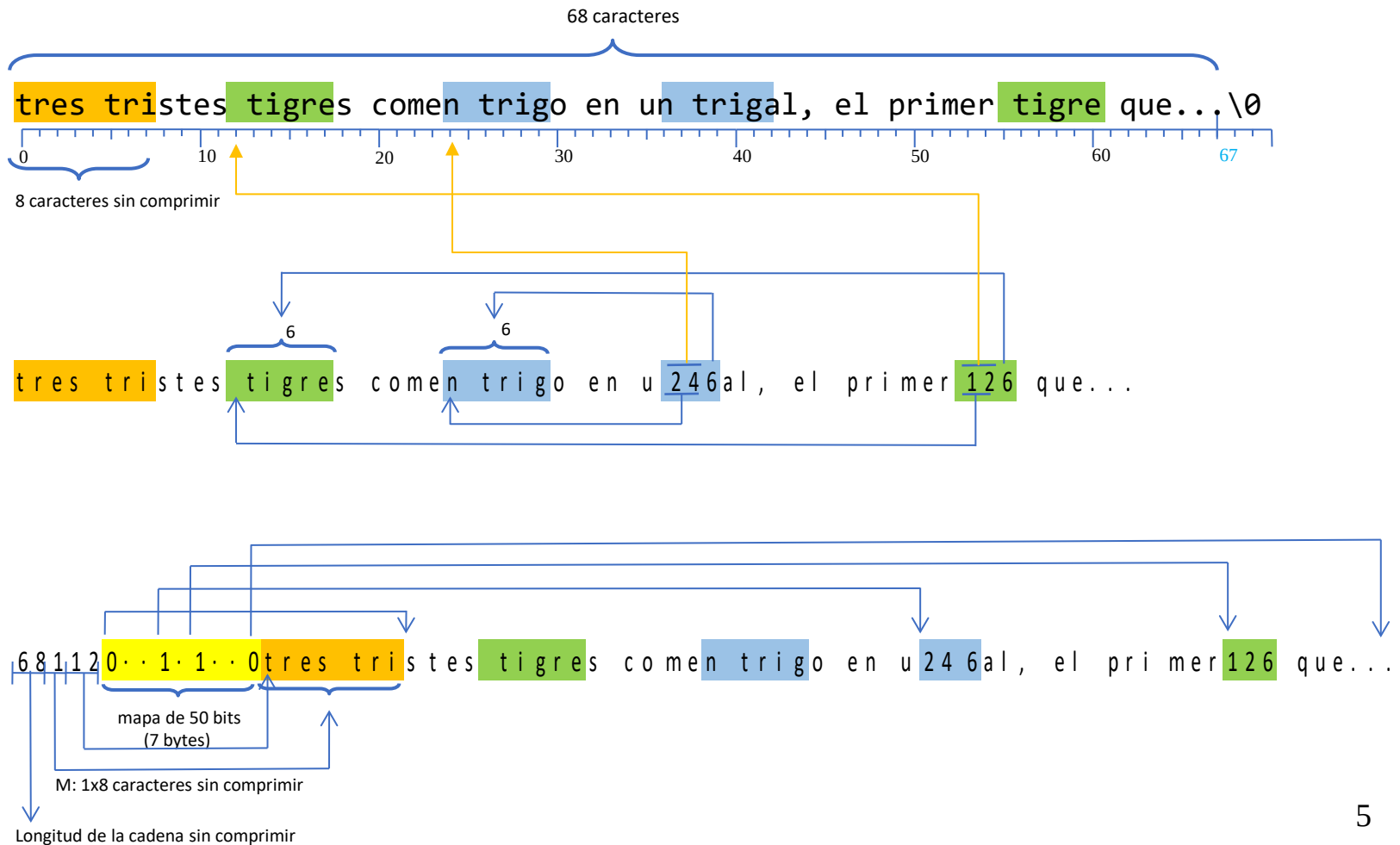


Texto comprimido:

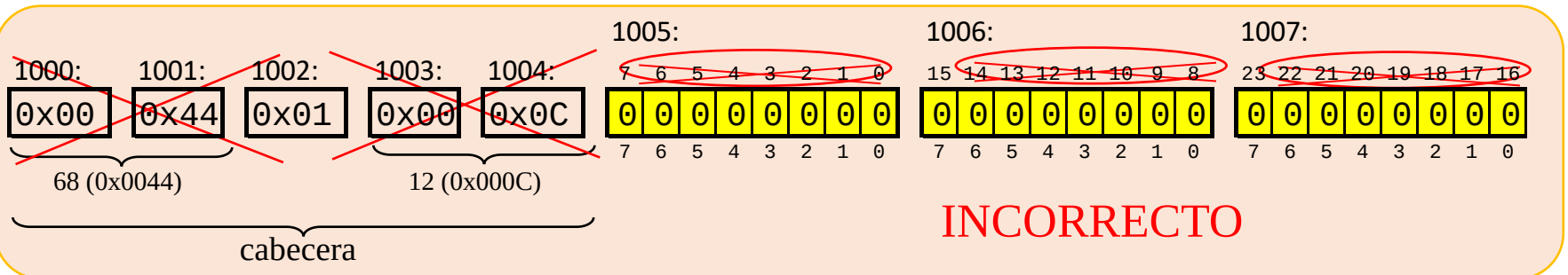
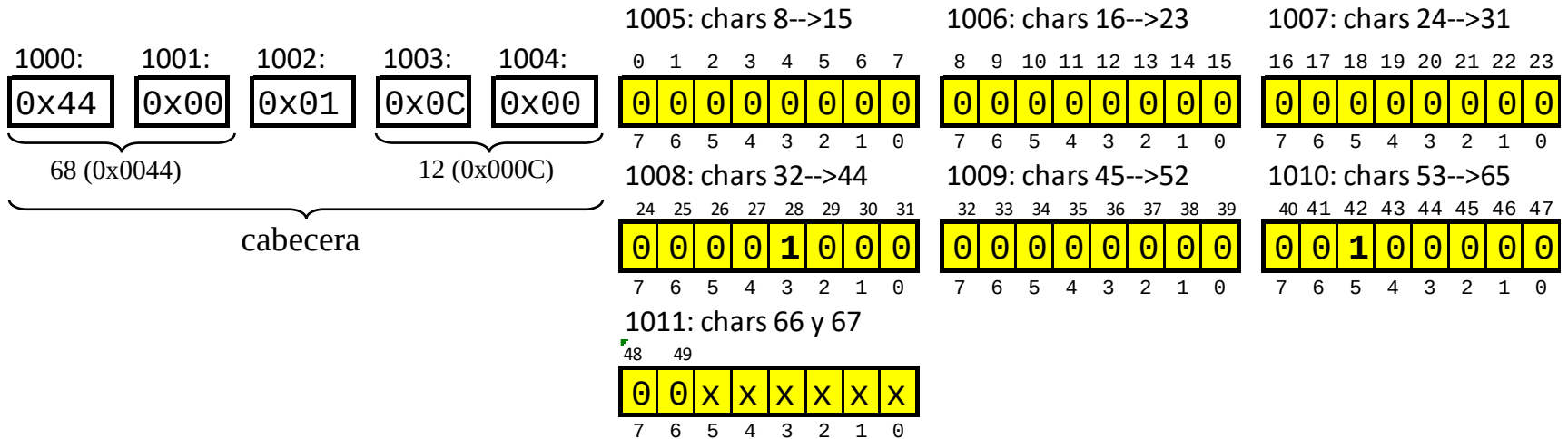


Compresión de texto

Ejemplo detallado de compresión en formato del proyecto, pero con $N=6$
(N = longitud mínima de una cadena de caracteres que se almacene por referencia)

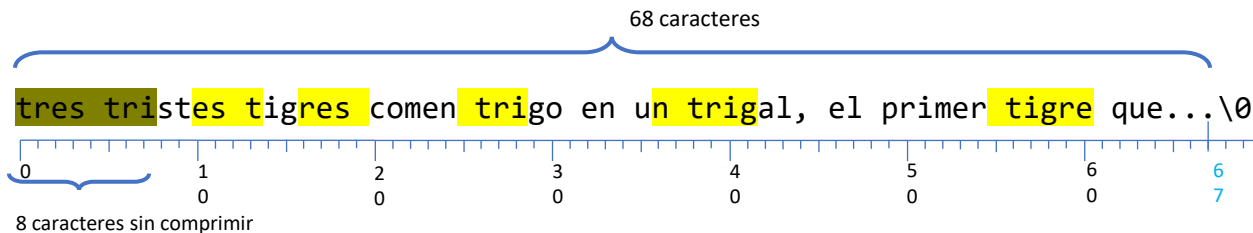


Mapa de bits (**N=6**)



Compresión de texto

Ejemplo detallado de compresión en el formato del proyecto, con $N=4$
 (N = longitud mínima de una cadena de caracteres que se almacene por referencia)



tres tristes tigres comen trigo en un trigal, el primer tigre que...\0	D:10	O:2	L:4
tres tristes tigres comen trigo en un trigal, el primer tigre que...\0	D:16	O:1	L:4
tres tristes tigres comen trigo en un trigal, el primer tigre que...\0	D:25	O:4	L:4
tres tristes tigres comen trigo en un trigal, el primer tigre que...\0	D:36	O:24	L:6
tres tristes tigres comen trigo en un trigal, el primer tigre que...\0	D:55	O:12	L:6

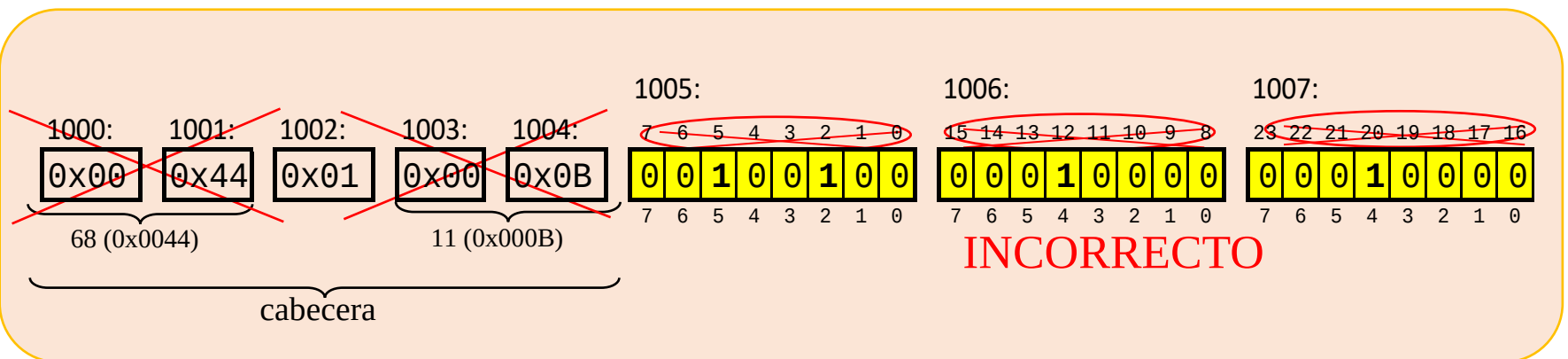
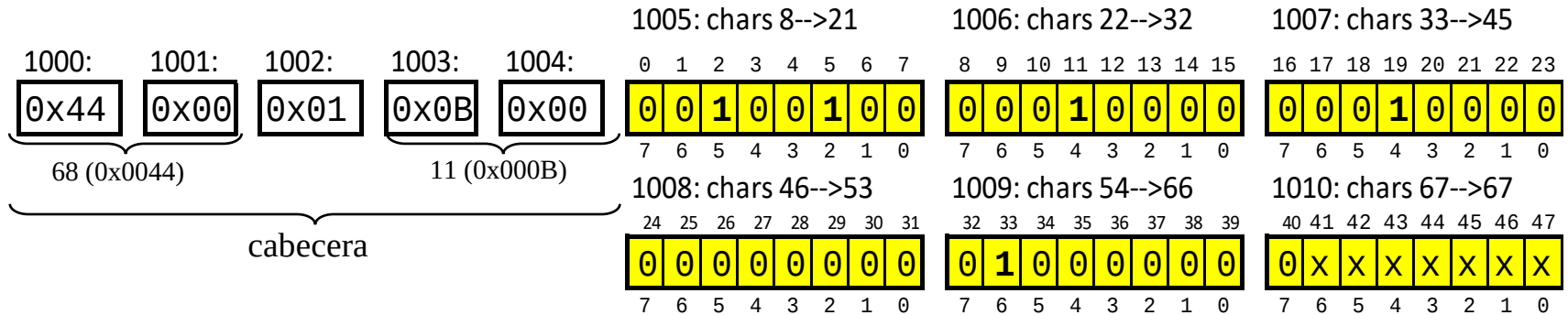
Leyenda: **Origen** **Destino** **D:destino, O:origen, L:longitud**

Mapa de bits (colores de la primera muestra: 0+2+1+2+1+5+1+7+1+13+1+7 = 41 bits → 6 Bytes

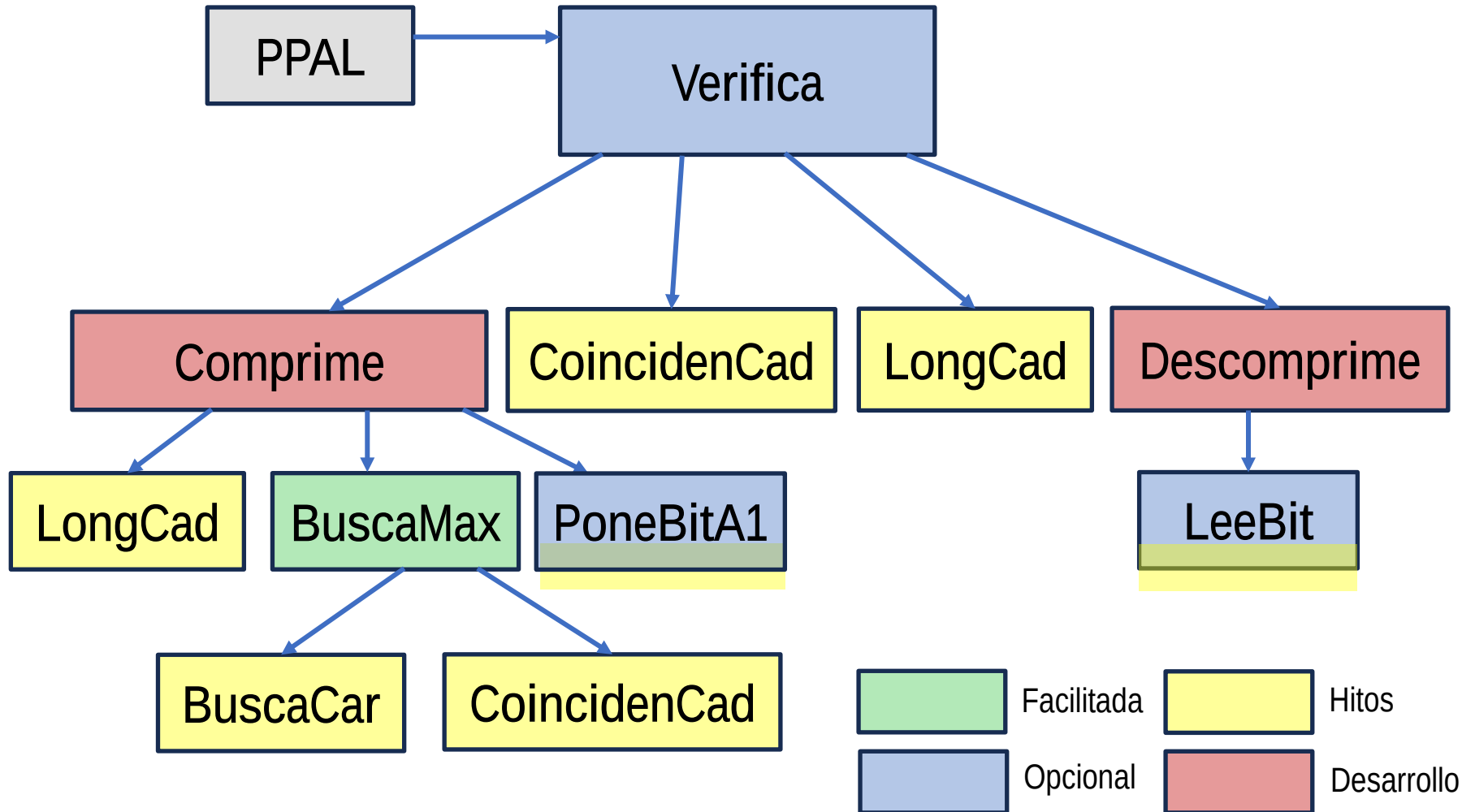
Cabecera: B0=low(68) B1=high(68) B2=1 B3=low(5+6) B4=high(5+6): 0x44 0x00 0x01 0x0B 0x00

Comienzo de la zona3 (caracteres/referencias): st-0₀l-0₀h-L-ig-0₁l-0₁h-L-comen-0₂l-0₂h-L-go...
 donde 0₀l-0₀h-L es 0x02 0x00 0x04, 0₁l-0₁h-L es 0x01 0x00 0x04, etc...

Mapa de bits (N=4)



Jerarquía de subrutinas



Longitud de una cadena

long = LongCad (cadena)

- Parámetros (en la pila):
 - cadena:** De entrada. Se pasa por dirección (32 bits sin signo). Es la cadena de caracteres cuya longitud se debe calcular.
- Valor de retorno:
 - long:** Se devuelve en **r29**. Es la longitud de la cadena sin incluir el carácter terminador **\0**. Es un entero (32 bits) sin signo (positivo o nulo).

Búsqueda de carácter

```
rv = BuscaCar (C, ref, from, to)
```

- Parámetros (en la pila):
 - C**: De entrada. Se pasa por valor. Es el carácter que se trata de localizar en la cadena **ref**.
 - ref**: De entrada. Se pasa por dirección. Es la dirección de comienzo de una cadena de caracteres.
 - from**: De entrada. Se pasa por valor. Es la posición de la cadena a partir de la que se debe comenzar a buscar.
 - to**: De entrada: Se pasa por valor. Es la posición de la cadena en la que debe finalizar la búsqueda.
- Valor de retorno:
 - rv**: Se devuelve en **r29**. Es la distancia al origen de **ref** de la primera ocurrencia del carácter **C** en el tramo que va de **Dir(ref[from])** a **Dir(ref[to-1])**. Si **C** no está en ese tramo, **rv** tomará el valor del parámetro **to**. Es un entero positivo o nulo.

Coincidencia de cadenas

```
long = CoincidenCad (cadena1, cadena2)
```

- Parámetros (en la pila):
 - cadena1:** De entrada. Se pasa por dirección: Es la primera de las dos cadenas de caracteres con que opera esta función.
 - cadena2:** De entrada. Se pasa por dirección: Es la segunda de las dos cadenas de caracteres con que opera esta función.
- Valor de retorno:
 - long:** Se devuelve en **r29**. Es el número de caracteres consecutivos que coinciden a partir del comienzo de sendas cadenas de caracteres. Es un entero que será nulo si el primer carácter de las cadenas es diferente y positivo en cualquier otro caso.

Búsqueda de la subcadena más larga (info)

```
rv = BuscaMax (ref, max, jj)
```

Esta subrutina se proporciona completa: su código está implementado.

Es necesaria para complementar el funcionamiento de **Comprime**.

Se apoya en **BuscaCar** y en **CoincidenCad** que tienen que estar implementadas

También sirve como muestra de una implementación válida de la subrutina cuyo comportamiento se describe en las próximas diapositivas.

Búsqueda de la subcadena más larga (1)

```
rv = BuscaMax (ref, max, jj)
```

- Parámetros (en la pila):

ref: De entrada. Se pasa por dirección. Es la cadena de caracteres en la que se busca la coincidencia más larga con cierta subcadena.

max: De entrada. Se pasa por valor. Es el desplazamiento desde el comienzo de **ref** donde se encuentra la subcadena que se trata de localizar en otra posición (más cercana al comienzo de **ref**).

jj: De salida. Se pasa por dirección. Es una variable entera en la que se devolverá el valor **-1** o el desplazamiento desde el comienzo de **ref** donde se encuentra la subcadena más larga que coincide con la que comienza en la dirección **Dir(ref[max])**.

- Valor de retorno:

rv: Se devuelve en **r29**. Es la longitud de la (primera) subcadena más larga encontrada que comience en el tramo **Dir(ref[0])** a **Dir(ref[max-1])** y coincida con la que comienza en **Dir(ref[max])**. Es un valor entero, positivo o nulo. En caso de ser nulo, en **jj** se devolverá el valor **-1**.

Búsqueda de la subcadena más larga (2)

```
rv = BuscaMax (ref, max, jj)
```

- Descripción:

La función busca en la cadena **ref**, desde su origen hasta la posición **max-1**, el tramo más largo de caracteres que **comenzando antes** de **Dir(ref[max])** coincidan con los que se encuentran a partir de **Dir(ref[max])**.

Proporciona, en la dirección que se pasa en el parámetro **jj**, la posición en la cadena **ref** de la subcadena más larga que coincide y en el valor de retorno, el número de caracteres coincidentes.

Si el número de caracteres coincidentes es **0**, la distancia que se devuelve en **jj** será **-1**.

Si la subcadena coincidente más larga se localiza en varias posiciones a partir de **ref**, se devolverá en **jj** el desplazamiento correspondiente a la primera de ellas.

La subcadena encontrada debe comenzar en una posición anterior a **max**, pero puede finalizar en una posición posterior.

Búsqueda de la subcadena más larga (3)

```
rv = BuscaMax (ref, max, jj)
```

- Algoritmo:
 1. Inicializa un marcador de posición **P** para recorrer **ref** y una variable con la longitud máxima encontrada hasta el momento **L**, ambos con valor 0. Asigna **-1** a **jj**.
 2. Recorre **ref** desde **P=0** hasta **max** o hasta encontrar una cadena de **255** caracteres:
 - Busca la siguiente posición del carácter **C=ref[max]** llamando a la subrutina **BuscaCar** con los parámetros: (**C, ref, P, max**) .
 - Si **BuscaCar** devuelve **max**, continúa en paso 3.
 - Avanza el marcador **P** hasta la posición devuelta por **BuscaCar**.
 - Llama a la subrutina **CoincidenCad(Dir(ref[P]), Dir(ref[max]))**
 - Si el valor devuelto por **CoincidenCad** es **255** o mayor, almacena la posición **P** en el parámetro de salida **jj**, actualiza **L (=255)** y continúa en el paso 3.
 - Si el valor devuelto por **CoincidenCad** es mayor que **L**, actualiza **L** y almacena la posición **P** en el parámetro de salida **jj** .
 - Avanza el marcador **P** hasta el siguiente carácter.
 3. Asigna a **r29** la longitud máxima final y retorna al llamante.

PoneBitA1 (info)

(void) PoneBitA1 (dirZonaCB, NumBit)

No es obligatorio implementar esta subrutina.

Sin embargo, su valoración de la parte de pruebas se verá limitada según se explica en el apartado de evaluación del enunciado

También sirve para verificar la validez de la implementación aportada para la subrutina **Comprime** si se ha desarrollado según el procedimiento descrito en esta presentación.

PoneBitA1 (1)

(void) PoneBitA1 (dirZonaCB, NumBit)

- Parámetros (en la pila):
 - dirZonaCB:** De entrada. Se pasa por dirección: Es el Campo de Bits del texto comprimido.
 - NumBit:** De entrada. Se pasa por valor: Es el número de bit a partir del comienzo de la zona que almacena el campo de bits que se quiere poner a 1.
- Valor de retorno:
No tiene.
- Descripción:

La función asigna el valor 1 al bit indicado en el parámetro **NumBit** a partir del comienzo de la zona **dirZonaCB**.
(El primer bit de la zona **dirZonaCB** es el **NumBit=0** y el resto de los bits identifican caracteres o referencias consecutivos).

PoneBitA1 (2)

(void) PoneBitA1 (dirZonaCB, NumBit)

- Procedimiento:

1. Se localizará el byte en que se encuentra el bit indicado con NumBit.
(Tenga en cuenta que esta localización se puede realizar mediante una operación muy sencilla).
2. Se leerá ese byte en un registro.
3. Se localizará el número de bit que se desea poner a 1 en ese byte.
(Tenga en cuenta que los bits del campo de bits se numeran siempre de forma creciente, del 0 en adelante, mientras que los bits de un byte, tal como se interpretan en este proyecto, se numeran empezando por el bit más significativo -bit 7- y terminan con el menos significativo -bit 0-).
4. Se pondrá a 1 el bit identificado en el paso anterior en el registro que contiene el byte seleccionado.
(Tenga en cuenta que esta operación se puede realizar con varias instrucciones lógicas o con una única instrucción de campo de bit sin más que preparar adecuadamente sus argumentos).
5. Se escribirá el registro que contiene el byte modificado en la dirección calculada en el primer paso.
6. Se devolverá el control al programa llamante.

Comprime (1)

rv = Comprime (texto, comprdo)

- Parámetros (en la pila):
 - texto:** De entrada. Se pasa por dirección: Es la cadena de caracteres que se ha de comprimir.
 - comprdo:** De salida. Se pasa por dirección: Es la zona de memoria en la que ha de quedar el texto comprimido.
- Valor de retorno:
 - rv:** Se devuelve en **r29**. Es el tamaño en bytes de la estructura que contiene el texto comprimido. Incluye la cabecera, el mapa de bits y el vector de caracteres o referencias. Es un entero positivo (32 bits).
- Descripción:

La función comprime el texto original almacenado a partir de **texto** y deja el resultado a partir de **comprdo**, de acuerdo al formato detallado en el enunciado de este proyecto.

Comprime (2)

```
rv = Comprime (texto, comprdo)
```

- Algoritmo:

1. Determinará la longitud de la cadena **texto** llamando a la subrutina **LongCad**.
2. Reservará espacio en una variable local (**pilaZona3**) para almacenar la secuencia de caracteres y referencias a cadenas del texto comprimido (la última parte del texto comprimido). Se reservará el número de bytes de **texto** ajustado por exceso a múltiplo de 4.
3. Inicializará las variables necesarias, por ejemplo, el desplazamiento para recorrer **texto**, la variable **pilaZona3**, o la que mantiene el nº de bit del mapa de bits según se va generando el texto comprimido (**nBit**).
4. Copiará directamente los **8xM** (M=1) caracteres iniciales de **texto** al comienzo de **pilaZona3**.
5. Recorrerá en un bucle los caracteres de **texto** hasta alcanzar su final:
 - a) Si el valor de **nBit** es múltiplo de ocho, se inicializará a 0 (0x00) el byte que contiene ese bit.

Comprime (3)

rv = Comprime (texto, comprdo)

- Algoritmo (cont.-1):

5.

....

- b) Localizará la siguiente subcadena repetida llamando a la subrutina **BuscaMax** con los parámetros **texto**, la posición actual del marcador que se usa para recorrer **texto** y la dirección de una variable local en la que se recogerá la posición de la subcadena, **Dir(P)**.
- c) Si la longitud **L** de la subcadena devuelta por **BuscaMax** es < 4 :
 - > Copiará el siguiente carácter de texto en la posición actual de **pilaZona3** y avanzará los punteros en una unidad.
 - > Incrementará en una unidad **nBit** (el número de bits del mapa de bits), el número de bytes de la zona de caracteres/referencias del texto comprimido y el número de caracteres de **texto** ya tratados.

Comprime (4)

rv = Comprime (texto, comprdo)

- Algoritmo (cont.-2):

5.

....

d) Si no, la longitud **L** de la subcadena es ≥ 4 :

> Llamará a **PoneBitA1** pasando el número de bit que corresponde.

> Copiará **P** y **L** en la posición actual de **pilaZona3**

> Avanzará el puntero que recorre **texto** en **L** unidades y el número de bytes de **pilaZona3** en 3 unidades.

> Incrementará en una unidad **nBit**.

6. Rellenará la cabecera de **comprdo**: **2** primeros bytes con longitud de **texto**; valor **1 (M)** en el tercer byte; **2** últimos bytes con suma de: nº de bytes de mapa de bits + 5 (bytes de esta cabecera).

7. Leerá los bytes del texto comprimido que se han escrito en la variable local **pilaZona3** y los copiará en **comprdo** a continuación del mapa de bits.

8. Retornará dejando en **r29** la suma del número de bytes del mapa de bits, más los que ocupa la zona del texto comprimido, más los 5 de la cabecera.

LeeBit (info)

```
rv = LeeBit (dirZonaCB, NumBit)
```

No es obligatorio implementar esta subrutina.

Sin embargo, su valoración de la parte de pruebas se verá limitada según se explica en el apartado de evaluación del enunciado

También sirve para verificar la validez de la implementación aportada para la subrutina **Descomprime** si se ha desarrollado según el procedimiento descrito en esta presentación.

LeeBit (1)

rv = LeeBit (dirZonaCB, NumBit)

- Parámetros (en la pila):
 - dirZonaCB:** De entrada. Se pasa por dirección: Es el Campo de Bits del texto comprimido.
 - NumBit:** De entrada. Se pasa por valor: Es el número de bit a partir del comienzo de la zona que almacena el campo de bits que se quiere poner a 1.
- Valor de retorno:
 - rv:** Se devuelve en **r29**. Es el valor del bit recogido en la posición indicada del mapa de bits. Es un entero (32 bits) que solo puede tomar los valores 0 ó 1.
- Descripción:

La función lee y devuelve el valor del bit indicado en el parámetro **NumBit** a partir del comienzo de la zona **dirZonaCB**.
(El primer bit de la zona **dirZonaCB** es el **NumBit=0** y el resto de los bits identifican caracteres o referencias consecutivos).

LeeBit (2)

rv = LeeBit (dirZonaCB, NumBit)

- Procedimiento:

1. Se localizará el byte en que se encuentra el bit indicado con NumBit.
(Tenga en cuenta que esta localización se puede realizar mediante una operación muy sencilla).
2. Se leerá ese byte en un registro.
3. Se localizará el número de bit dentro de ese byte cuyo valor se desea leer.
(Tenga en cuenta que los bits del campo de bits se numeran siempre de forma creciente, del 0 en adelante, mientras que los bits de un byte, tal como se interpretan en este proyecto, se numeran empezando por el bit más significativo -bit 7- y terminan con el menos significativo -bit 0-).
4. Se leerá el bit identificado en el paso anterior en el registro que contiene el byte seleccionado.
(Tenga en cuenta que esta operación se puede realizar con varias instrucciones lógicas o con una única instrucción de campo de bit sin más que preparar adecuadamente sus argumentos).
5. Se devolverá en r29 un 0 o un 1 en consonancia con el valor del bit seleccionado.

Descomprime (1)

rv = Descomprime (com, desc)

- Parámetros (en la pila):
 - com:** De entrada. Se pasa por dirección: Es la zona de memoria en la que se encuentra el texto comprimido.
 - desc:** De salida. Se pasa por dirección: Es la cadena de caracteres en la que ha de quedar el texto una vez descomprimido.
- Valor de retorno:
 - rv:** Se devuelve en **r29**. Es la longitud de la cadena que contiene el texto una vez que se ha descomprimido. Es un entero (32 bits) positivo o nulo.
- Descripción:

La función descomprime el texto que tiene el formato detallado en el enunciado de este proyecto y está almacenado a partir de **com** dejando el resultado a partir de **desc**.

Descomprime (2)

rv = Descomprime (com, desc)

- Algoritmo:

1. Inicializará las variables necesarias, por ejemplo, los punteros o desplazamientos para recorrer **com** y **desc**, o el mapa de bits.
2. Copiará de **com** a **desc** los **8xM** caracteres iniciales que no se comprimen, avanzando sus punteros o marcadores.
3. Recorrerá en un bucle los bytes de **com** hasta alcanzar su final. En cada iteración del bucle llamará a **LeeBit** con los parámetros correspondientes y:
 - a) Si el valor devuelto por **LeeBit** es **0**:

Copiará el siguiente carácter de **com** a **desc** y avanzará sus punteros.
 - b) Si no (el valor devuelto por **LeeBit** es **1**):

Copiará los **L** caracteres situados a partir de **Dir(desc[P])** a la posición que marca el puntero actual de **desc**, incrementando este en **L** unidades y el puntero a **com** en 3 unidades.
 - c) Avanzará el puntero a la posición del siguiente bit.
4. Añadirá el terminador “\0” al final de **desc**.
5. Retornará dejando en **r29** la longitud del texto.

Verifica (info)

```
rv = Verifica(texto, Long1, Long2)
```

No es obligatorio implementar esta subrutina.

Sin embargo, su valoración de la parte de pruebas se verá limitada según se explica en el apartado de evaluación del enunciado

También sirve para verificar la validez de la implementación aportada para las subrutinas **Comprime** y **Descomprime**.

Verifica (1)

```
rv = Verifica(texto, Long1, Long2)
```

- **Parámetros (en la pila):**
 - texto:** De entrada. Se pasa por dirección: Es la cadena de caracteres que se ha de comprimir, descomprimir y verificar.
 - Long1:** De salida. Se pasa por dirección: Es la variable en la que la subrutina escribirá la longitud del texto original que se pasa en el primer parámetro.
 - Long2:** De salida. Se pasa por dirección: Es la variable en la que la subrutina escribirá la longitud del texto obtenido tras comprimir y después descomprimir el texto original que se pasa en el primer parámetro.
- **Valor de retorno:**
 - rv:** Se devuelve en r29. Es el resultado global de la verificación: **0** si la verificación ha sido correcta, **-1** si las longitudes difieren o **-2** si difieren en contenido. Es un entero que solo puede tomar los valores **0, -1 o -2**.

Verifica (2)

rv = Verifica(texto, Long1, Long2)

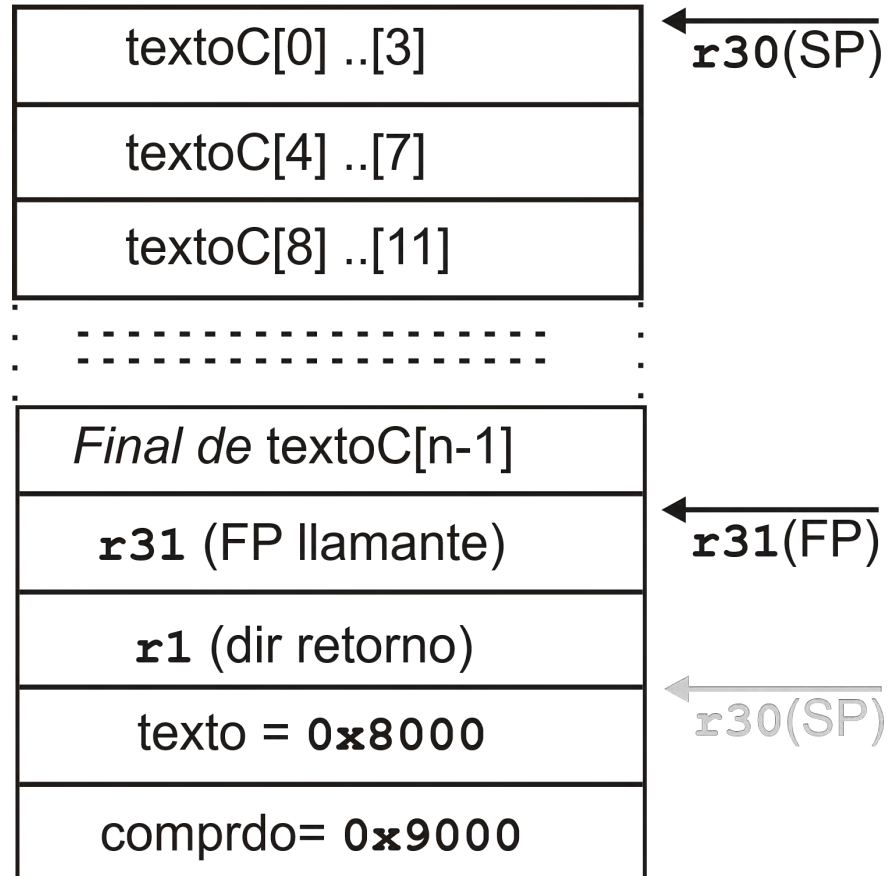
- Algoritmo:

1. Reservará dos zonas de la pila con tamaño suficiente para almacenar respectivamente el texto comprimido (PilaCom) y el texto descomprimido (PilaDes):
 - a. Se calculará el tamaño que ocuparía el texto comprimido en el caso peor, es decir, si no se encuentra ninguna subcadena de longitud mayor o igual a 4. PilaCom reservará una zona de ese tamaño una vez ajustado por exceso al primer múltiplo de 4.
 - b. Se reservará para PilaDes el tamaño ocupado por el texto original (incluyendo su terminador) una vez ajustado por exceso al primer múltiplo de 4.
2. Llamará a Comprime con los parámetros (texto, PilaCom)
3. Llamará a Descomprime con los parámetros (PilaCom, PilaDes)
4. Si la longitud del texto original, texto, es diferente a la del texto descomprimido, PilaDes, devolverá el control poniendo rv=-1.
5. Si el valor devuelto por CoincidenCad(texto, PilaDes) es igual a la longitud del texto original, asignará rv=0; en caso contrario asignará rv=-2. Devolverá el control al llamante.

Tratamiento de la pila

Comprime:

```
PUSH    (r1)
PUSH    (r31)
addu    r31, r30, r0
ld      r10, r31, 8
.....
```



Ejemplo: Búsqueda de un carácter (1)

```
org      0x8000
C:      data "_"
REF:    data "AAAABBBBCCCCDDDEEEE_Fin\0"
from:   data 4
to:     data 24
```

```
88110> v 0x8000 12
```

32768	5F000000	41414141	42424242	43434343
32784	44444444	45454545	5F46696E	00000000
32800	04000000	18000000	00000000	00000000

Ejemplo: Búsqueda de un carácter (2)

```
org    0x8400
ppal:  or    r30, r0, 0x9000
      or    r31, r30, r30
                                   ; Alternativamente se puede
                                   ; poner:
LOAD   (r11, C)      ; or    r11, r0, 0x5F ; "_"
LEA    (r10, REF)
LOAD   (r12, from)   ; or    r12, r0, 4
LOAD   (r13, to)     ; or    r13, r0, 24

PUSH   (r13) ; to
PUSH   (r12) ; from
PUSH   (r10) ; REF
PUSH   (r11) ; C
bsr    BuscaCar
ret:   addu   r30, r30, 16
      stop
```

Ejemplo: Búsqueda de un carácter (3)

r30=36848 (0x8FF0)

Direcciones de memoria:

36848 5F000000 04800000 04000000 18000000

Resultado:

r30=36848 (0x8FF0) r29=20 (0x14)

Direcciones de memoria:

36848 5F000000 04800000 04000000 18000000

Programas de prueba

LEA: MACRO...

org xxx

D1: data...

D2: data...

D3: res...

org yyy

PPAL: "inicialización de la pila"

...

"paso de parámetros (PUSH)"

...

bsr subrutina

"vaciado de la pila (POP)".

stop

Algunas normas de implementación

➤ Asignación de etiquetas y memoria

- Es necesario mantener sin cambios el punto de entrada a **todas** las subrutinas: *LongCad*, *BuscaCar*, *CoincidenCad*, *Comprime* y *Descomprime*. También a *BuscaMax* (proporcionada), y *PoneBitA1*, *LeeBit* y *Verifica* (opcionales).
- La implementación debe realizarse evitando utilizar las direcciones **0x00000000** a **0x0000000F** así como la **0x00010000** y todas las superiores a esta, que se reservan para el código y datos que se utiliza en las correcciones

➤ Avisos importantes

- Todas las subrutinas deberán finalizar su ejecución con el mismo valor en el puntero de pila que había al principio de la subrutina (justo antes de ejecutar su primera instrucción).
- Siempre que una subrutina necesite almacenar temporalmente información, usará memoria de la pila y nunca variables definidas en direcciones absolutas.

➤ Contenido de CDV25.ens

- Se requiere que todas las subrutinas que componen el proyecto estén debidamente comentadas, cada una junto con un programa principal y los datos de prueba. Si para una entrega aún no se ha desarrollado una subrutina, por ejemplo, **Verifica**, se definirá:

Verifica: **jmp (r1)**

Algunas normas de implementación

➤ Contenido de la memoria (**memoria.pdf**)

Entre 1 y 2 páginas de cada uno de los siguientes apartados:

- a. Resumen diario de fechas/horas empleadas para la elaboración del proyecto por cada uno de los autores. Indicación explícita del n° total de horas invertido por cada autor.
- b. Resumen de casos de prueba empleados, resaltando los que difieran de los aportados por el dpto.
- c. Observaciones finales y comentarios personales, incluyendo la siguiente declaración firmada por cada autor:

Yo, (nombre y apellido), declaro **conocer y manifiesto mi conformidad** con las siguientes normas sobre copia y fraude académico:

- el uso para el desarrollo del proyecto de cualquier material de otros grupos/alumnos o generado automáticamente con herramientas informáticas, está explícitamente prohibido, independientemente del motivo alegado.
- el departamento comprobará la copia entre todas las entregas de todos los grupos que intervienen en la convocatoria y considerará copia cuando aparezca esa condición en cualquiera de las entregas analizadas.
- el departamento realizará finalmente un análisis global de posibles casos de copia, incluyendo la inspección visual de implementaciones de los grupos cuyo código presente un mayor índice de coincidencias. Igualmente, se comprobarán algunas implementaciones elegidas al azar.

Entrega (fechas – febrero 2026)

El plazo de entrega del proyecto estará abierto para darse de alta y para entregar código y memoria:

Desde el miércoles 22 de octubre hasta el jueves día 18 de diciembre de 2025

Cada grupo podrá utilizar las siguientes correcciones:

- **Primera corrección:** martes 28/10/2025 21:00
2 correcciones adicionales: 29-31 de octubre y 3-7 y 11 de noviembre
- **Hito 1 evaluable:** miércoles 12/11/2025:
subrutinas **LongCad**, **BuscaCar** y **CoincidenCad**: hasta 1 punto
2 correcciones adicionales: 13-14 y 17-20 de noviembre
- **Hito 2 evaluable:** viernes 21/11/2025:
subrutinas **PoneBitA1** y **LeeBit**: hasta 1 punto
4 correcciones adicionales: 24-28 de noviembre y 1-5, 9-12 y 15-17 de diciembre
- **Última corrección:** jueves 18/12/2025 21:00

- Todas las correcciones se realizarán a partir de las 21:00.
- Para solicitar la corrección de un día concreto basta con entregar correctamente los ficheros del proyecto antes de las 21:00 de esa fecha
- El examen del proyecto está planificado para día del examen final de la asignatura

Entrega (fechas – julio2026)

El plazo de entrega del proyecto estará abierto para darse de alta y para entregar código y memoria:

Desde el miércoles día 27 de mayo hasta el miércoles día 24 de junio de 2026.

Cada grupo podrá utilizar las siguientes correcciones:

· **Primera corrección:** lunes 01/6/2026 21:00

6 correcciones adicionales: 2-5, 9-12, 16-19 y 22-23 de junio de 2026

· **Hito 1 evaluable:** lunes 8/6/2026:

subrutinas **LongCad**, **BuscaCar** y **CoincidenCad**: hasta 1 punto

· **Hito 2 evaluable:** lunes 15/6/2026:

subrutinas **PoneBitA1** y **LeeBit**: hasta 1 punto

· **Última corrección:** miércoles 24/6/2026 21:00

- Todas las correcciones se realizarán a partir de las 21:00.
- Para solicitar la corrección de un día concreto basta con entregar correctamente los ficheros del proyecto antes de las 21:00 de esa fecha.
- El examen del proyecto está planificado para día del examen final de la asignatura.

Entrega (contenido)

Ficheros:

1. **autores:** Es un fichero ASCII que solo se entrega en el momento de registrarse en el gestor de prácticas. Deberá contener los apellidos, nombre, número de matrícula, DNI y dirección de correo electrónico de los autores del proyecto. El proyecto se realizará individualmente o en grupos de dos alumnos. Cada línea de este fichero contendrá los datos de uno de los autores de acuerdo al siguiente formato:

Nº Matrícula; DNI; apellido apellido, nombre; correo electrónico

NOTA: La palabra-clave seleccionada en el momento de hacer el registro será compartida entre los miembros del grupo, por lo que no se debe utilizar una clave habitual de ninguno de ellos.

2. **CDV25.ens:** Subrutinas e información necesaria para las pruebas. Véase el enunciado.
3. **memoria.pdf:** *Memoria-resumen*, en formato PDF y tamaño Din-A4, en cuya portada deberá figurar claramente el nombre y apellidos de los autores del proyecto, el identificador del grupo de alumnos y el nombre de la asignatura. Véase la descripción completa en el enunciado.

Evaluación (2025/2026)

- El proyecto consta de tres partes: código y memoria (*código-memoria*), pruebas de funcionamiento (*pruebas*) y examen del proyecto (*examen*). Para superar el proyecto se deberá obtener la calificación de apto en cada una de las tres partes.
- Pruebas: para que un grupo pueda optar al examen del proyecto será necesario obtener al menos 5 puntos en la última corrección (o con anterioridad). La puntuación de esta parte es la siguiente:
 - *Hitos evaluables*. Puntuación proporcional al número de pruebas de cada hito superadas en la fecha establecida para el mismo:
 - Convocatoria ordinaria 12/11 y 21/11 de 2025; extraordinaria 8/6 y 15/6 de 2026.
 - Hito1: Subrutinas LongCad y BuscaCar y CoincidenCad (hasta 1 punto).
 - Hito2: Subrutinas PoneBitA1 y LeeBit (hasta 1 punto).
 - *Subrutinas*. Puntuación proporcional al número de pruebas superadas:
 - Obligatorias: Descomprime, hasta 3 puntos; Comprime, hasta 3 puntos
 - Opcionales: PoneBitA1 hasta 0,5 puntos; LeeBit hasta 0,5 puntos
 - Opcional: Verifica, hasta 1 punto.
 - *Condición obligatoria*. Cada una de las subrutinas Comprime y Descomprime debe superar al menos 3 de sus pruebas para que la puntuación obtenida como la suma de las partes *Hitos evaluables* y *Subrutinas* pueda alcanzar el mínimo de 5 puntos necesario para optar al examen.
- Examen: A partir de 3 puntos hace media con la nota de pruebas.
- Código-memoria: ± 1 punto en función de la calidad. “No apto” si no cumple requisitos básicos. Véase la sección “Avisos importantes” en la página 23 del enunciado.

Evaluación (2025/2026)

- Nota de la asignatura:

$$\text{Nota_asignatura} = 0,8 \times \text{Nota_Teoría} + 0,2 \times \text{Nota_Proyecto}$$

- Calificación final del proyecto:

- Si (pruebas ≥ 5 puntos) Y (examen ≥ 3 puntos) Y (código-memoria Apto):
 $\text{Nota_Proyecto} = 0,7 \times \text{Nota_pruebas} + 0,3 \times \text{Nota_examen} \pm 1$ (código-memoria)
- Si (pruebas < 5 puntos) $\rightarrow$ no se hace examen y:
 $\text{Nota_Proyecto} = \text{Nota_pruebas}/3$
- Si (examen < 3 puntos):
 $\text{Nota_Proyecto} = \text{Nota_examen}$ (del proyecto)
- Si (código-memoria es “No apto”):
 $\text{Nota_Proyecto} =$ hasta 1 punto (dependiendo de las causas y las pruebas superadas)